



Università degli Studi di Bergamo

Dipartimento di Ingegneria

Corso di Laurea in Ingegneria Informatica

Anno Accademico 2025/2026

GESTIONE

Ingegneria del Software

The Scouting Suite

Brivio Andrea — Matricola: 1089359

Fischetti Alessandro — Matricola: 1080491

Pesenti Lorenzo — Matricola: 1072635

Indice

1	Introduzione	2
2	Software Life Cycle	2
2.1	Tipo di processo di sviluppo adottato	2
2.2	Confronto con il Project Plan	2
2.3	Organizzazione degli sprint	2
2.4	Formalizzazione UML del processo	3
2.5	Approccio Model-Driven Development	3
2.6	Caratteristiche di Software Product Line	3
3	Configuration Management	4
3.1	Strumenti e piattaforma	4
3.2	Workflow di sviluppo	4
3.3	Project Board e monitoraggio	4
3.4	Metriche e statistiche	4
4	People Management and Team Organization	5
4.1	Composizione e struttura del team	5
4.2	Organizzazione del lavoro	5
4.3	Ruoli e responsabilità	5
4.4	Comunicazione e coordinamento	5
4.5	Gestione dei rischi	6
5	Strumenti e metodologie	6
5.1	Tool utilizzati	6
5.2	Metodologie di sviluppo	6
6	Conclusioni	6
6.1	Lessons Learned	7

1 Introduzione

Il presente documento descrive gli aspetti di **gestione del progetto** relativi allo sviluppo del software *The Scouting Suite*. Il sistema è un'applicazione completa per l'analisi e lo scouting calcistico, sviluppata con architettura Spring Boot + Vaadin.

Vengono analizzati i seguenti aspetti chiave:

- il ciclo di vita del software adottato (Prototipazione Evolutiva),
- le strategie di configuration management con GitHub,
- l'organizzazione del team e la gestione delle attività,
- gli strumenti e le metodologie utilizzate.

Il documento integra quanto descritto nel *Project Plan*, evidenziando le scelte effettive durante lo sviluppo.

2 Software Life Cycle

2.1 Tipo di processo di sviluppo adottato

Il progetto *The Scouting Suite* è stato sviluppato seguendo un approccio di **Prototipazione Evolutiva combinato con Sviluppo Incrementale**, adatto alla complessità del dominio calcistico e all'incertezza iniziale sui requisiti.

Le caratteristiche principali:

- **Prototipo iniziale:** Sistema minimo funzionante con funzionalità core
- **Incrementi funzionali:** Aggiunta iterativa di feature complesse
- **Refactoring continuo:** Ristrutturazione guidata da metriche di qualità

2.2 Confronto con il Project Plan

Il processo seguito presenta alcune differenze significative rispetto alla pianificazione iniziale:

- **Maggior enfasi sul design architetturale:** Scelta di Spring Boot + Vaadin invece di tecnologie più semplici
- **Estensione della modellazione UML:** Realizzazione di 8 tipi di diagrammi UML completi
- **Adozione di pattern avanzati:** Implementazione di Factory, Strategy e Specification patterns
- **Riorganizzazione temporale:** Maggior tempo dedicato alla fase di testing e integrazione

2.3 Organizzazione degli sprint

Il lavoro è stato organizzato in **sprint bisettimanali** con le seguenti fasi principali:

Sprint	Attività principali
Sprint 1 (Ottobre)	Analisi requisiti, studio dominio calcistico, diagrammi Use Case iniziali
Sprint 2 (Novembre)	Design architetturale, Class Diagram, Package Diagram, setup progetto
Sprint 3 (Dicembre)	Implementazione core: Player Entity, Repository, servizi base
Sprint 4 (Dicembre)	Sviluppo UI Vaadin, implementazione filtri dinamici, testing
Sprint 5 (Dicembre)	Refactoring, ottimizzazione performance, documentazione finale

Tabella 1: Organizzazione degli sprint di sviluppo

2.4 Formalizzazione UML del processo

Il processo di sviluppo è stato documentato tramite diagrammi UML specifici:

- **Activity Diagram:** Flusso ETL di importazione dati CSV
- **Sequence Diagram:** Interazioni durante l'applicazione dei filtri
- **State Machine Diagram:** Stati del sistema durante la ricerca
- **Communication Diagram:** Collaborazione tra componenti

2.5 Approccio Model-Driven Development

Il progetto adotta parzialmente i principi del **Model-Driven Development (MDD)**:

- **Modelli come artefatti primari:** Diagrammi UML completi prima dell'implementazione
- **Tracciabilità:** Collegamento esplicito tra requisiti → design → codice
- **Coerenza mantenuta:** Allineamento continuo tra modelli e implementazione
- **Generazione codice:** Utilizzo parziale di Papyrus per generazione scheletro classi

2.6 Caratteristiche di Software Product Line

Il sistema presenta caratteristiche di **Software Product Line (SPL)** che ne facilitano l'estensione futura:

- **Variabilità funzionale:** Filtri base vs. avanzati, colonne configurabili
- **Architettura modulare:** Layer separati (Presentation, Business, Persistence)
- **Estensibilità:** Pattern Factory e Strategy per aggiunta nuovi tipi di filtro
- **Configurabilità:** Interfaccia utente personalizzabile per diversi ruoli (scout, analista, utente base)

3 Configuration Management

3.1 Strumenti e piattaforma

Per la gestione della configurazione è stato utilizzato **GitHub** come piattaforma principale, seguendo le best practice descritte nel Capitolo 4 del testo di riferimento.

Stack tecnologico per il CM:

- **Repository:** GitHub (git)
- **Client:** GitHub Desktop + Git CLI
- **Issue Tracking:** GitHub Issues
- **CI/CD:** GitHub Actions (setup base)

3.2 Workflow di sviluppo

Il progetto ha adottato un workflow **GitFlow semplificato**:

1. **Feature branching:** Ogni nuova funzionalità sviluppata in branch separato
2. **Pull Request:** Revisione codice obbligatoria prima del merge
3. **Code Review:** Controllo di qualità e conformità agli standard
4. **Merge su main:** Solo dopo superamento di test automatici

3.3 Project Board e monitoraggio

È stata utilizzata una **Project Board GitHub** in stile Kanban per il monitoraggio delle attività:

Colonna	Descrizione
Backlog	Requisiti e feature da implementare
To Do	Attività pianificate per lo sprint corrente
In Progress	Attività in fase di sviluppo
Review	Codice pronto per revisione
Done	Attività completate e verificate

Tabella 2: Organizzazione della Project Board

3.4 Metriche e statistiche

Le principali metriche di configuration management:

- **Commit totali:** 150+ (con messaggi descrittivi)
- **Branch:** 12+ branch per feature
- **Issues:** 25+ issue tra bug, enhancement, task
- **Pull Requests:** 15+ con review sistematica
- **Release tag:** v1.0 per la versione finale

4 People Management and Team Organization

4.1 Composizione e struttura del team

Il team di progetto è composto da tre membri con competenze complementari:

Membro	Ruoli principali e competenze
Andrea Brivio	Architettura Spring Boot, design pattern, testing
Alessandro Fischetti	UI Vaadin, frontend, UX/UI design
Lorenzo Pesenti	Database design, JPA, modellazione UML

Tabella 3: Competenze e ruoli del team

4.2 Organizzazione del lavoro

L'organizzazione segue un modello **collaborativo con specializzazioni**, ispirato alle strutture descritte nel Capitolo 5:

- **Responsabilità condivise:** Decisioni architetturali collettive
- **Specializzazione per layer:** Divisione naturale per competenze
- **Pair programming:** Per componenti critiche (Specification Factory)
- **Code ownership collettivo:** Tutti i membri conoscono l'intero codice

4.3 Ruoli e responsabilità

- **Analisi e modellazione UML:** Creazione e manutenzione diagrammi
- **Design architetturale:** Scelta pattern e tecnologie
- **Implementazione backend:** Servizi Spring, repository, business logic
- **Sviluppo frontend:** Componenti Vaadin, UI/UX
- **Testing e qualità:** Unit test, integration test, analisi statica
- **Documentazione:** Javadoc, documenti progetto, presentazioni

4.4 Comunicazione e coordinamento

La comunicazione è stata gestita attraverso multipli canali:

- **Riunioni sincrone:** Meeting settimanali per allineamento
- **Chat asincrona:** Gruppo WhatsApp/Telegram per comunicazioni rapide
- **Condivisione documenti:** Google Drive per documentazione
- **Repository GitHub:** Issue tracking e project board per tracciamento
- **Tool di modellazione:** Condivisione file Papyrus e PlantUML

4.5 Gestione dei rischi

Principali rischi identificati e mitigati:

- **Complessità tecnologica:** Formazione su Spring Boot e Vaadin
- **Dimensione dataset:** Ottimizzazione query e paginazione
- **Integrazione componenti:** Testing intensivo e mock objects
- **Tempistiche:** Buffer temporale e prioritizzazione requisiti

5 Strumenti e metodologie

5.1 Tool utilizzati

Categoria	Strumenti
IDE	Eclipse for Java Developers con plugin Papyrus
Modellazione	Papyrus UML, PlantUML per generazione automatica
Version Control	GitHub, GitHub Desktop
Build	Apache Maven
Testing	JUnit 5, Mockito, Spring Boot Test
Analisi codice	SonarLint per qualità del codice
Database	H2 (sviluppo), preparazione per PostgreSQL
Documentazione	LaTeX, Google Docs, Maven Site

Tabella 4: Stack tecnologico completo

5.2 Metodologie di sviluppo

- **Test-Driven Development:** Per componenti critiche (servizi, repository)
- **Continuous Integration:** Setup base con GitHub Actions
- **Code Review sistematica:** Ogni PR esaminata da almeno un altro membro
- **Refactoring guidato da metriche:** Basato su analisi SonarLint
- **Pair programming:** Per algoritmi complessi (query dinamiche)

6 Conclusioni

La gestione del progetto *The Scouting Suite* ha seguito un approccio strutturato ma flessibile, adattandosi alle specificità del dominio calcistico e alle sfide tecniche emerse.

I punti chiave del successo gestionale:

- **Processo adattivo:** Prototipazione evolutiva per gestire l'incertezza
- **Configuration management efficace:** GitHub con workflow strutturato

- **Team organizzato:** Competenze complementari e comunicazione efficiente
- **Qualità sistematica:** Testing, code review, analisi statica
- **Documentazione completa:** UML, Javadoc, documenti gestionali

Il progetto dimostra l'applicazione pratica dei principi di Ingegneria del Software in un contesto reale, bilanciando rigore metodologico e pragmatismo nello sviluppo.

6.1 Lessons Learned

- L'investimento iniziale in design architetturale paga in fase di sviluppo
- La modellazione UML dettagliata riduce i misunderstanding tra team
- I pattern appropriati (Factory, Strategy) facilitano l'estensibilità
- Il testing automatico è essenziale per componenti con dipendenze complesse
- La comunicazione frequente previene divergenze nel design